



Instituto Tecnológico de Las Américas (ITLA)

Asignatura:

Programación II – TDS-006

Docente:

Francis Ramírez

Sustentante:

Joel Alberto Benitez Varela 2025-1049

Adrian Francisco Brito Nelkitts 2025-1150

Nombre del proyecto:

AeroVeloz

Tema:

Tarea - Elaboración del Documento de Arquitectura del Proyecto

República Dominicana, 01 de febrero de 2026.

1. Introducción

La Arquitectura del Sistema de Información de Vuelos (SIV) se documenta en el presente SAD, estableciéndose como la referencia técnica principal para los equipos de desarrollo, arquitectura y control de calidad. Este documento traduce los requisitos operativos en estructuras de software concretas, definiendo las decisiones de diseño y los patrones necesarios para la construcción del sistema.

El núcleo de la solución se fundamenta en el ecosistema .NET (C#) bajo una estricta Arquitectura en Capas (Layered Architecture). Este enfoque estratégico permite un desacoplamiento efectivo entre la lógica de negocio y la persistencia de datos, garantizando un sistema mantenible, escalable y robusto, donde la integridad de la información y la trazabilidad de las operaciones son la prioridad absoluta.

1.1. Propósito

El propósito de este documento es brindar una visión técnica profunda de AeroVeloz, sirviendo como el plano de construcción oficial para los ingenieros de software. A diferencia de documentos funcionales, este apartado detalla el "cómo" de la implementación: desde la organización de clases y la estrategia de inyección de dependencias, hasta los protocolos de comunicación entre capas y el manejo de eventos internos mediante delegados nativos.

Su objetivo final es guiar la implementación técnica para asegurar que los servicios se orquesten de manera transaccional y eficiente, garantizando que la información crítica de los vuelos se procese con consistencia y llegue de forma proactiva a los usuarios finales.

1.2. Descripción del sistema

El SIV es una solución tecnológica diseñada para centralizar y automatizar la difusión de información sobre el estado de los vuelos en un entorno aeroportuario.

El sistema se desarrolla principalmente bajo el patrón **Model-View-Controllers (MVC)** en C#, separando la lógica de presentación de la lógica de negocio.

Su diseño permite la interacción de dos perfiles de usuario principales:

- **Administrador del Sistema:** Rol técnico encargado exclusivamente de la gestión de accesos, creación de usuarios y asignación de permisos de seguridad. No interviene en la operación de vuelos.

- **Equipo de Operaciones:** Personal encargado de la gestión diaria del aeropuerto, incluyendo la creación de vuelos, asignación de puertas, actualización de estados y manejo de incidencias operativas.
- **Usuarios Externos (Pasajeros y Visitantes):** Acceden a una interfaz de consulta para visualizar información en tiempo real, limitada a un rango de tiempo específico para optimizar el rendimiento.

El *objetivo* central es eliminar la desincronización informativa, asegurando que cualquier cambio en la operación de un vuelo se refleje de manera inmediata y proactiva en todos los canales de difusión.

1.3. Alcance del proyecto

1.3.1. Decisiones de Alcance

A continuación, se especifican ciertas funcionalidades que forman parte de esta etapa del desarrollo, así como las exclusiones que permiten determinar el diseño técnico inicial:

Funcionalidades incluidas:

- **Gestión del Ciclo de Vida de Vuelos:** Creación y actualización de vuelos (número, origen, destino, puerta y hora programada).
- **Control de Estados Operativos:** Implementación de la lógica para transiciones de estado (A tiempo, Retrasado, Cancelado, entre otros.).
- **Módulo de Trazabilidad y Auditoría:** Registro obligatorio de cada cambio realizado, cumpliendo con el requisito de inmutabilidad de los logs.
- **Sistema de Difusión de Notificaciones:** Generación automática de avisos ante los cambios de estado para mantener informados a los usuarios.
- **Interfaz de Consulta para Usuarios Externos:** Filtro de visualización restringido a las 48 horas (pasadas y futuras) respecto a la hora del servidor.
- **Arquitectura de Capas en C#:** Separación clara entre la capa de Persistencia (Base de Datos), la capa de Dominio (Lógica) y la capa de Presentación (MVC).

Funcionalidades excluidas:

- **Venta de Boletos y Reservas:** El sistema es simplemente informativo, no procesa transacciones financieras ni selección de asientos.
- **Gestión de Equipaje:** No se incluye el rastreo de maletas ni la asignación de bandas de reclamo en esta fase.

- **Control de Personal de Tripulación:** No se gestionan roles de pilotos o azafatas, enfocándose únicamente en el objeto "Vuelo".
- **Seguridad Física del Aeropuerto:** El sistema no tiene conexión con cámaras o controles de acceso biométrico.

Funcionalidades postergadas:

- **Integración con APIs de Radares Externos:** La actualización automatizada mediante servicios de terceros se planea para una versión 2.0.
- **Aplicación Móvil Nativa:** El acceso se creará vía web responsive, dejando el desarrollo para iOS y Android como una fase futura.
- **Kioscos de Autoservicio:** La implementación en hardware físico especializado dentro de la terminal se evaluará tras la estabilidad de la plataforma web.

1.4. Público objetivo

Este documento está diseñado para ser consultado por los siguientes interesados:

- **Desarrolladores:** Para entender la estructura de carpetas y capas en C#.
- **Arquitectos:** Para verificar la viabilidad del patrón MVC y la arquitectura propuesta.
- **Testers (QA):** Para saber qué flujos de datos deben probar con mayor importancia, especialmente en la trazabilidad y estados de vuelo.

2. Vista general de la arquitectura

2.1 Vista general del sistema

El Sistema de Información de Vuelos (SIV) ha sido construido con un diseño en capas y modular, implementando el patrón **MVC (Model-View-Controller)** en C# para asegurar una clara división de responsabilidades, posibilitar la evolución futura del sistema y hacer más fácil su mantenimiento. Esta arquitectura define una separación estructural entre la capa de presentación, la capa de aplicación, la capa de dominio y la capa de persistencia, garantizando que las modificaciones en una capa no afecten directamente a las otras. Así, se fomenta la estabilidad operativa del sistema, así como la posibilidad de escalar y reutilizar sus componentes.

El SIV centraliza la información de vuelos y se comporta como una única fuente confiable de datos, posibilitando que los diferentes actores tengan acceso a datos actualizados y consistentes en tiempo real, disminuyendo así las inconsistencias y errores comunicativos en el ambiente aeroportuario.

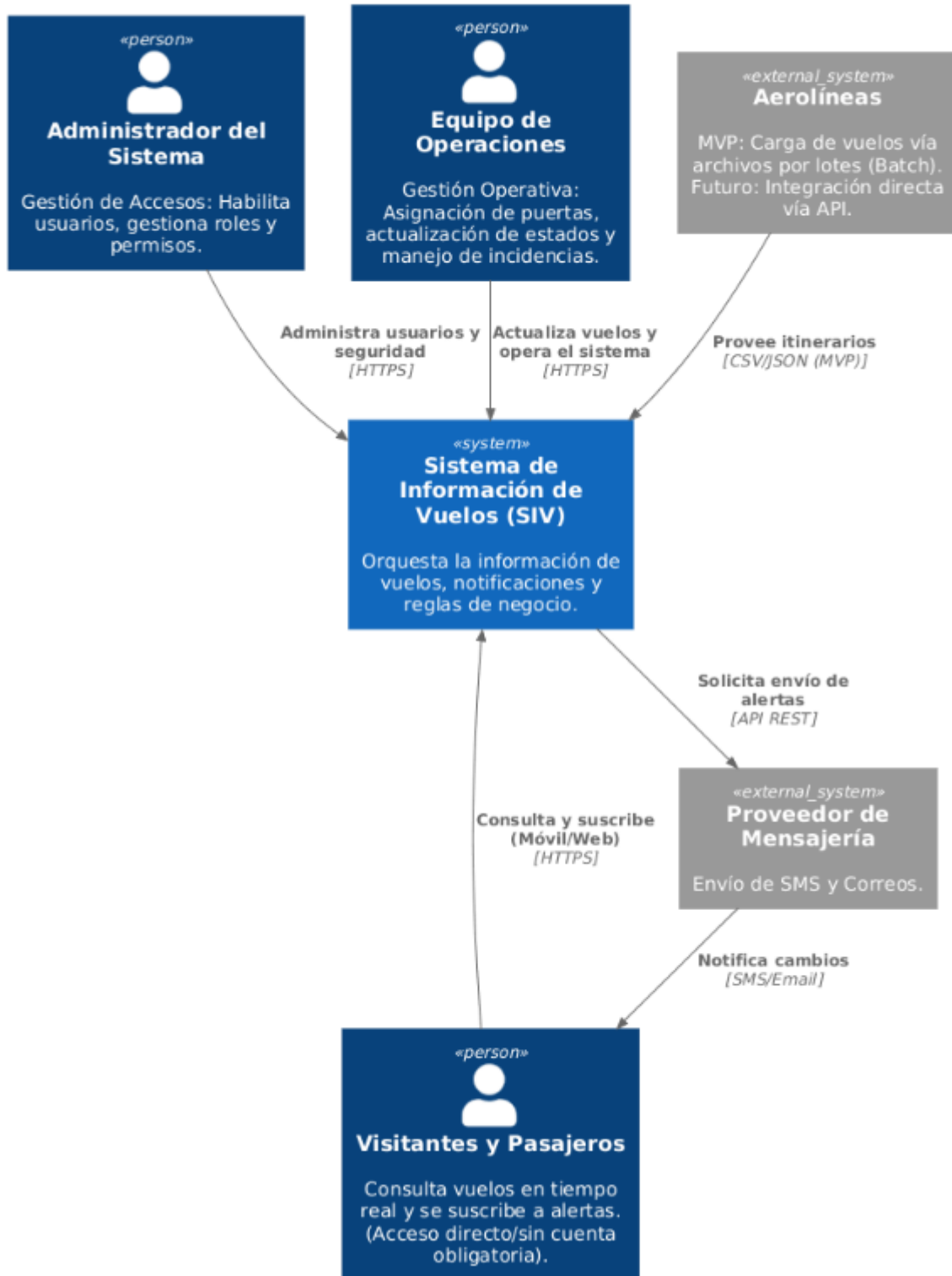
2.1. Diagramas de alto nivel

Para representar la arquitectura del SIV de forma integral, se utilizan dos niveles de abstracción según el modelo C4

Nivel 1: Diagrama de Contexto

Este diagrama representa la visión más general del sistema, identificando las fronteras entre el **SIV** y su entorno. Muestra cómo el sistema interactúa con los actores y sistemas externos sin entrar en detalles tecnológicos de implementación.

Diagrama de Contexto - Sistema de Información de Vuelos (SIV)

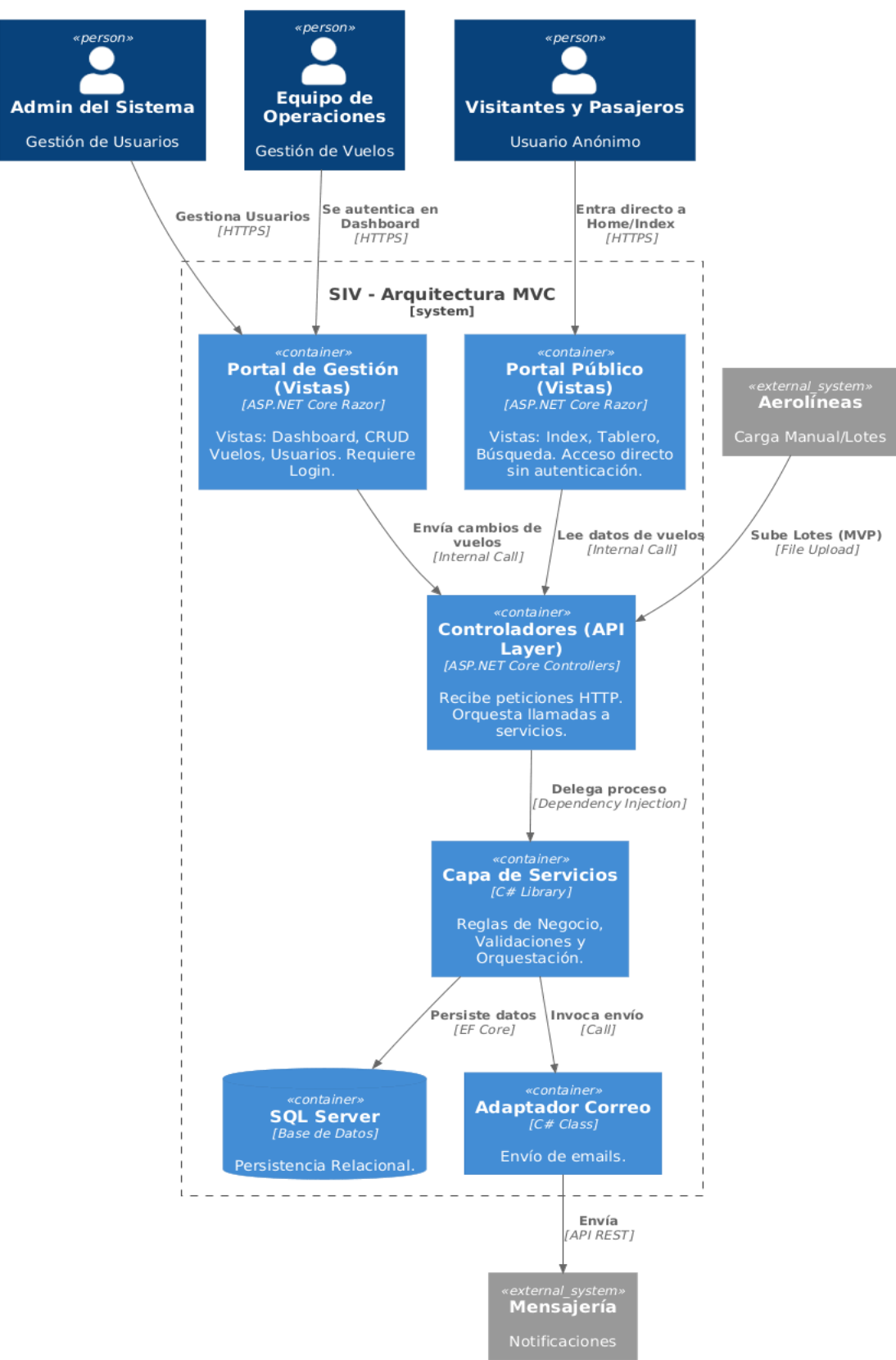


SIV (Caja Central): Actúa como el núcleo centralizado encargado de procesar y difundir la información de vuelos.

Actores: El **Administrador de Operaciones** gestiona la carga de datos, mientras que el **Usuario Externo** consume la información y se suscribe a alertas personalizadas.

Sistemas Externos: Se integran las **Aerolíneas** como fuentes primarias de datos y el **Proveedor de Mensajería** como el canal de salida para las notificaciones proactivas.

Nivel 2: Diagrama de Contenedores (Arquitectura técnica)



El sistema implementa una Segregación de Interfaces para optimizar el flujo de cada perfil de usuario:

1. **Acceso de Visitantes y Pasajeros (Portal Público):** Acceden directamente a las vistas públicas de consulta (Tablero de Vuelos) sin necesidad de autenticación previa. Estas solicitudes son manejadas por controladores de lectura optimizados para alto tráfico.
2. **Acceso del Equipo de Operaciones (Portal de Gestión):** Interactúan a través de un área segura que requiere autenticación estricta para realizar operaciones de escritura (CRUD de Vuelos).

Lógica de Orquestación (Backend): Ambas interfaces convergen en la Capa de Aplicación, donde los Servicios actúan como orquestadores centrales. En lugar de utilizar eventos complejos, estos servicios ejecutan la lógica de negocio de manera secuencial y explícita:

- Primero, validan las reglas de negocio en la Capa de Dominio.
- Luego, invocan a los Repositorios para persistir los datos de forma transaccional.
- Finalmente, y solo si el guardado fue exitoso, invocan directamente al Servicio de Notificaciones para alertar a los usuarios.

Este enfoque elimina la dependencia de eventos de dominio distribuidos, garantizando un flujo de control lineal, fácil de auditar y libre de inconsistencias.

2.2. Principios arquitectónicos

La arquitectura del SIV se basa en los principios arquitectónicos que a continuación se detallan, y que orientan tanto la implementación técnica como las decisiones de diseño del sistema.

División de responsabilidades

Cada capa del sistema desempeña un papel específico y claramente definido. La presentación se restringe a la interacción con el usuario, la aplicación organiza los casos de uso, las reglas de negocio están en el dominio y la persistencia se encarga del almacenamiento de datos. Esta división simplifica el mantenimiento y disminuye el acoplamiento.

Inmutabilidad y trazabilidad

Cualquier modificación significativa en el sistema produce un registro obligatorio que contiene el

actor, la fecha y la acción llevada a cabo. Los registros de auditoría son inalterables, lo que posibilita la reconstrucción del historial total de sucesos operativos y el cumplimiento de las exigencias de supervisión y control.

Concordancia de la información

El sistema funciona como una fuente centralizada de información, asegurando que todos los canales de difusión y usuarios tengan acceso a datos actualizados y coherentes en tiempo real.

Capacidad de escalar y mantener

El diseño modular posibilita que el sistema se expanda de forma gradual, añadiendo integraciones externas o funciones nuevas sin comprometer la estabilidad del núcleo.

Seguridad lógica y control de acceso

Los perfiles de usuario limitan el acceso a las funcionalidades. Los usuarios externos pueden acceder solamente a consultas informativas, lo que asegura el principio del privilegio mínimo, mientras que los administradores tienen privilegios de gestión.

Optimización del rendimiento

Para proteger el desempeño del sistema, la información disponible para usuarios externos se limita a un rango temporal de 48 horas (pasadas y futuras), disminuyendo la carga sobre la base de datos y mejorando ciertos tiempos de respuesta.

2.3. Patrones utilizados

La arquitectura del SIV tiene en cuenta patrones establecidos en la ingeniería de software, los cuales se han escogido por su madurez y pertinencia para el contexto del sistema:

Model–View–Controller (MVC)

Permite separar el modelo de datos, la lógica de presentación y el control del flujo, lo que simplifica el mantenimiento y la organización del código.

Arquitectura por capas

Fomenta un diseño desacoplado y escalable al estructurar el sistema en niveles bien definidos: presentación, aplicación, dominio y persistencia.

Service Layer (Capa de servicios de aplicación)

Coordina las operaciones entre la persistencia, los servicios externos y el dominio centralizando la implementación de los casos de uso del sistema.

Repository Pattern

Desacoplando la lógica de negocio de los pormenores de persistencia y brindando más adaptabilidad frente a modificaciones tecnológicas, abstrae el acceso a la base de datos.

Data Transfer Object (DTO)

Para transmitir información entre capas, se emplean objetos de transferencia de datos, con el fin de no exponer las entidades de dominio directamente y así aumentar la seguridad y la claridad del modelo.

Service Orchestration: (Orquestación de Servicios): Se implementa un modelo de orquestación explícita donde los Servicios de Aplicación actúan como coordinadores centrales del flujo de negocio. En lugar de disparar eventos dispersos, los servicios gestionan las transacciones invocando secuencialmente a los componentes necesarios (Repositorios, Notificadores, Auditoría).

Audit Trail

Posibilita la supervisión, el control y el análisis posteriores de sucesos al garantizar que se registre todo lo que ocurre de manera crítica en el sistema.

3. Vistas arquitectónicas

3.1 Vista lógica

Descripción:

Esta vista describe la organización funcional del Sistema de información de vuelos (SIV) y cómo se distribuyen sus responsabilidades en **capas, módulos y componentes** para garantizar:

1. Una fuente centralizada y consistente de información de vuelos
2. Recepción/registro de vuelos y cambios desde aerolíneas
3. Gestión de estados y trazabilidad histórica

4. Seguimiento informativo por usuarios externos sin cuenta

5. Generación y envío de notificaciones solo a suscritos

6. Registro/auditoria de todos los eventos relevantes

Dicha organización será llevada a cabo empleando el patrón de diseño MVC, una arquitectura en capas (Layered Architecture) y una serie de tecnologías entre las cuales se encuentran:

1. Razor asp.net core como capa de presentación.
2. Asp.net Core Rest API como backend de aplicación.
3. OneSignal como medio de transporte de las notificaciones o alertas.
4. SignalR como medio de actualización y sincronización en tiempo real entre los diversos aplicativos del SIV.
5. SQL Server como persistencia (vuelos, estados, cambios, suscripciones, notificaciones, auditoría).

Componentes funcionales

El sistema de SIV está estructurado en capas principales que siguen las reglas del modelo MVC y la arquitectura en capas (Layered Architecture) cada una encargándose de una lógica y parte del sistema en concreto:

1. **Capa Presentación (Razor)** - gestiona la lógica UI de conexión con los usuarios del sistema siendo así responsable de
 - 1.1 Consultas de vuelos (públicas) y vistas consistentes.
 - 1.2 Pantallas operativas (cambios, correcciones, supervisión)
 - 1.3 Flujo de “seguir vuelo” sin cuenta mediante una suscripción que se genere para el usuario (captura de datos mínimos):
 1. Identificador del vuelo
 2. Canal de contacto (email, sms o token push)
2. **Api (Asp.net Core Rest Api)** - Esta capa es la responsable de exponer los endpoints Rest para:
 1. Consulta pública/operativa de vuelos
 2. Recepción de lotes de aerolíneas
 3. Gestión de cambios/estados.

4. Suscripciones/seguimientos
5. Auditoria y supervisión

3. **Aplicación (Servicios de la aplicación / Casos de uso)** - Aquí vive la orquestación de casos de uso

1. Validar datos mínimos de vuelos (programación).
2. Procesar lotes, detectar incongruencias y alertar a Operaciones.
3. Aplicar cambios operativos y transicionales estados.
4. Registrar historial (inmutable) de estados/cambios/notificaciones.
5. Disparar notificaciones por eventos relevantes.

4. **Dominio**

1. Estados permitidos y transiciones
2. Tipos mínimos de cambios operativos
3. Un único estado vigente por vuelo
4. Seguimiento informativo que no altera la operación
5. Notificar solo a suscritos al vuelo y registrar trazabilidad

5. **Infraestructura y Persistencia (SQL server + integraciones)**

1. Repositorios SQL server
2. Adaptador a sistema de mensajería (SMS/Email/Push)
3. Logs/auditoria inmutable
4. Control de acceso por roles y visibilidad

Módulos funcionales

El SIV está estructurado en una serie de módulos, los cuales encapsulan funciones clave:

Presentación:

- FlightsController -> controlador de los elementos pertinentes a los vuelos provenientes de x aerolínea.
- OperationsController -> controlador del flujo de procesos realizados por operaciones
- SubscriptionsController -> gestor de las listas de suscripciones de los vuelos pertinentes para envíos y supervisión para actores de operaciones.
- AuditController -> controlador para los elementos de auditoría.
- Views/DTOs -> elemento que maneja las vistas UI de la aplicación y los objetos de transferencia de datos.

Aplicación:

- FlightQueryService -> servicio que actúa como consulto de la data de estados de los vuelos.
- OperationsAppService -> servicio que consulta y conecta con los cambios operarios.
- AirlinesIntegrationService -> elemento que procesa los lotes de vuelos.
- SubscriptionsService -> elementos que se encarga de manejar y controlar las listas de suscripciones a vuelos de interés.
- NotificationsService -> elemento que provee el mensaje pertinente para cada usuario.
- **AuditService** -> proveedor de los elementos que se encargan de los procesos de auditoría de la aplicación.

Dominio:

- addFlight (flight, state, history, rules)
- AddSubscriptions
- addNotifications
- TransitionsRules/Policy

Infraestructura:

- Repository (FlightRepo, SubscriptionsRepo, NotificationsRepo, AuditRepo)
- Adapters (AirlineAdapter, SmsAdapter, EmailAdapter, PushAdapter)
- Messages/jobs, DB

Transversal:

- AuthN/AuthZ, monitoring

Servicios principales:

En este apartado se encuentran los servicios principales del SIV basados tanto en los servicios de la aplicación, el dominio y los adaptadores de infraestructura.

1. **FlightQueryService:** Supervisar/consulta los cambios ocurridos en los vuelos provenientes de la aerolínea, devuelve lista/detalle de vuelos con estado vigente, horarios y datos visible según el actor, permite aplicar filtros típicos (fecha, aerolínea, numero de vuelo, origen/destino, estado).
2. **AirlineBatchIngestionService:** Se encarga de recibir los lotes de vuelos provenientes de las aerolíneas, registrando los vuelos o actualizando programación base, normaliza los datos (formatos, fechas, identificadores), envía cada ítem del lote al validador y al “pipeline” de registro.
3. **BatchValidationService:** Servicio que valida que los lotes cumplan con los requisitos mínimos para operar, marcando registros en caso de fallos como incongruentes.
4. **OperationalChangeService:** Se encarga de manejar los cambios por parte de los usuarios operacionales (cambios de puertas, incongruencias en lotes de vuelos, etc), dispara actualización del estado vigente y el flujo de notificaciones.
5. **FlightStateMachineService:** Servicio permite que los estados cambien de forma válida (transiciones permitidas), garantizando un único estado vigente.

Clases

Las diversas capas de la arquitectura y módulos contienen las múltiples clases que permitirán al sistema funcional de forma correcta, entre las cuales están:

- **Presentación:**

Presentación.ApiController:

1. flightsController:
 - a. GetFlights (FlightFilterDto filter): PagedResult<FlightDto>
 - b. GetFlightDetail (Guid flightId): FlightDetailDto
2. auditController:

- a. GetFlightHistory (Guid flightId): FlightHistoryDto
- b. SearchAudit (AuditQueryDto query): PagedResult<AuditEntryDto>

- **DTOs**

- 1. DTOs.Flights

- 1.FlightsFilterDto

- a. DateTime? Date
- b. string? AirlineCode
- c. string? Flight Number
- d. string? Origin
- e. string? Destination
- f. FlightState? State
- g. int Page
- h. int PageSize

- 2.. FlightDto

- a. Guid FlightId
- b. string FlightNumber
- c. string AirlineCode
- d. string Origin
- e. string Destination
- f. DateTime ScheduledTime
- g. FlightState CurrentState2

- **Aplicación:**

- Aplicacion.Subscription

- 1. ISubscriptionService: interface

- a. Guid Subscribe (Guid flightId, SubscriptionChannel channel, string contactValue)
- b. void Cancel (Guid subscriptionId)
- c. List<Subscription> GetByFlight (Guid flightId)
- d. List<Guid> GetFollowedFlightsByContact (SubscriptionChannel channel, string contactValue)

- 2. SubscriptionService: ISubscriptionService

- a. _repo: ISubscriptionRepository
- b. _policy: ISubscriptionPolicy

- **Domain**

- Domain.Subscription

- 1. Subscription: class

- a. id: Guid
- b. flightId: Guid
- c. channel: SubscriptionChannel
- d. active: bool
- e. createdAt: DateTime
- f. canceledAt: ¿DateTime?
- g. Cancel (Datetime at): void.

2. SubscriptionPolicy: ISubscriptionPolicy

- a) CanSubscribe (Guid flightId, SubscriptionChannel channel, string contactValue):
bool

• Infraestructura

Infraestructura. Subscription

1. ISubscriptionRepository: interface

- a) Add (Subscription subscription): void.
- b) Cancel (Guid subscriptionId): void.
- c) GetActiveByFlight (Guid flightId): List<Subscription>
- d) GetActiveByContact (SubscriptionChannel channel, string contactValue):
List<Subscription>

2. SqlSubscriptionRepository: ISubscriptionRepository

- a) SivDbContext

Diagrama de clases

Capa de presentación:

- 1. [Diagrama de clases capa de presentacion.pdf](#)

Capa de aplicación:

- 1. [Diagrama de clases capa de aplicacion parte 1.pdf](#)
- 2. [Diagrama de clases capa de aplicacion parte 2.pdf](#)
- 3. [Diagrama de clases capa de aplicacion parte 3.pdf](#)
- 4. [Diagrama de clases capa de aplicacion parte 4.pdf](#)
- 5. [Diagrama de clases capa de aplicacion parte 5.pdf](#)

Capa de dominio:

- 1. [Diagrama de clase capa de dominio parte 1.pdf](#)

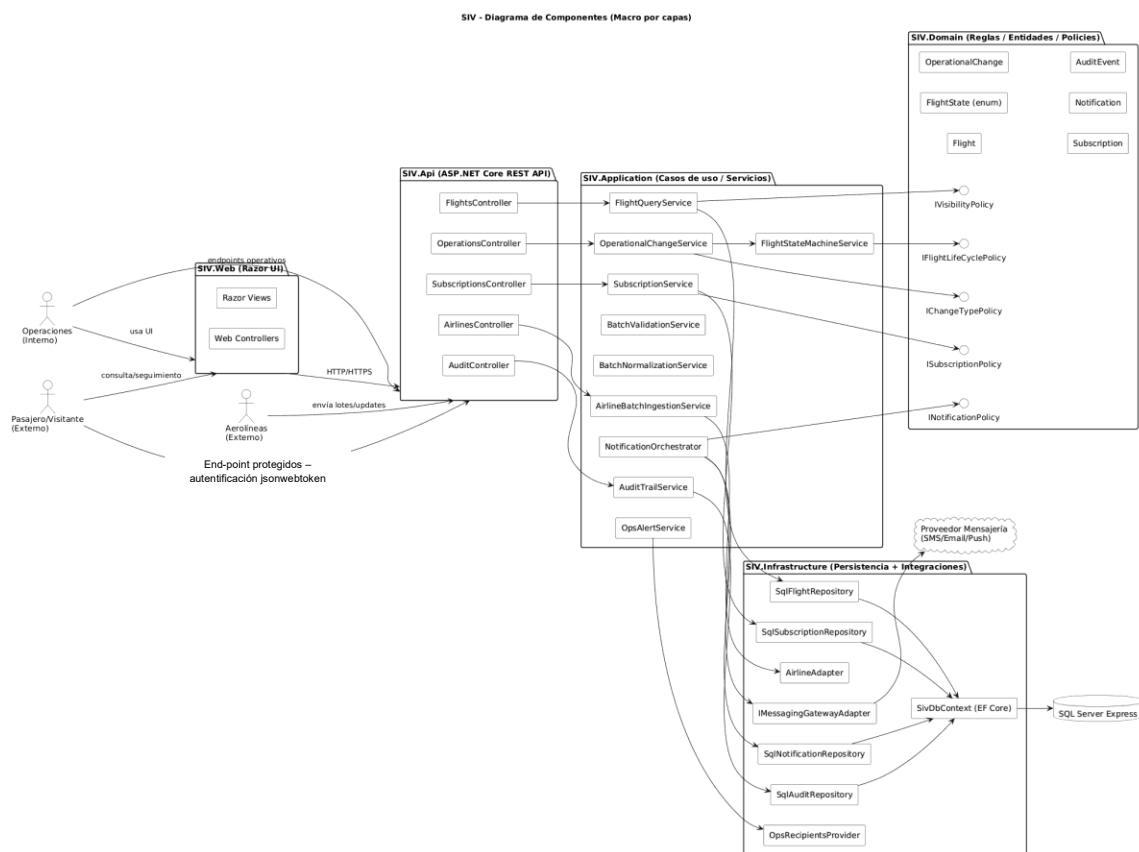
2. [Diagrama de clase capa de dominio parte 2.pdf](#)
3. [Diagrama de clases capa de dominio parte 3.pdf](#)
4. [Diagrama de clases capa de dominio parte 4.pdf](#)

Capa de infraestructura

1. [Diagrama de clase capa de infraestructura parte 1.pdf](#)
2. [Diagrama de clases capa de infraestructura parte 2.pdf](#)

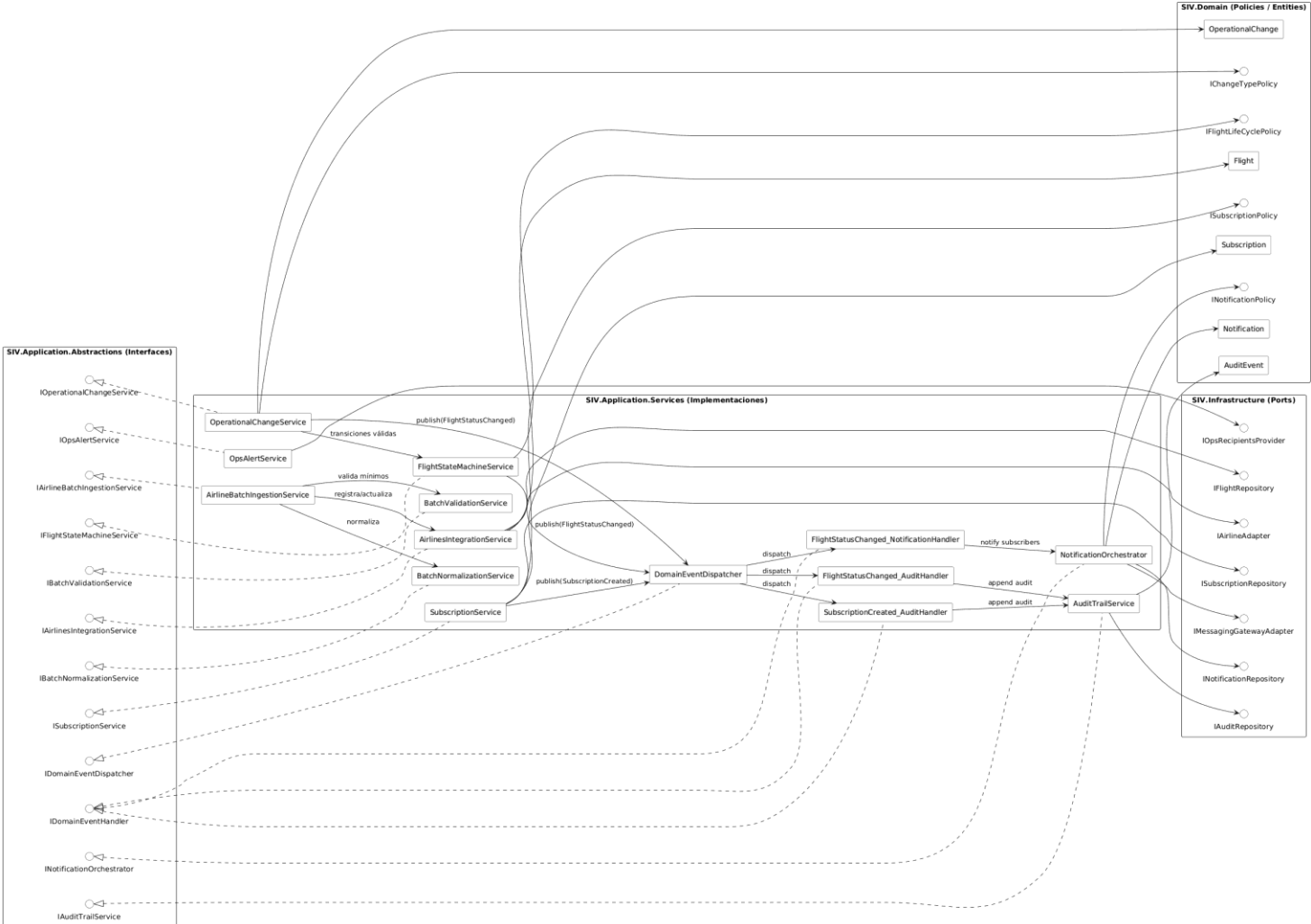
Diagrama de componentes:

Nivel 3. Diagrama de componentes:

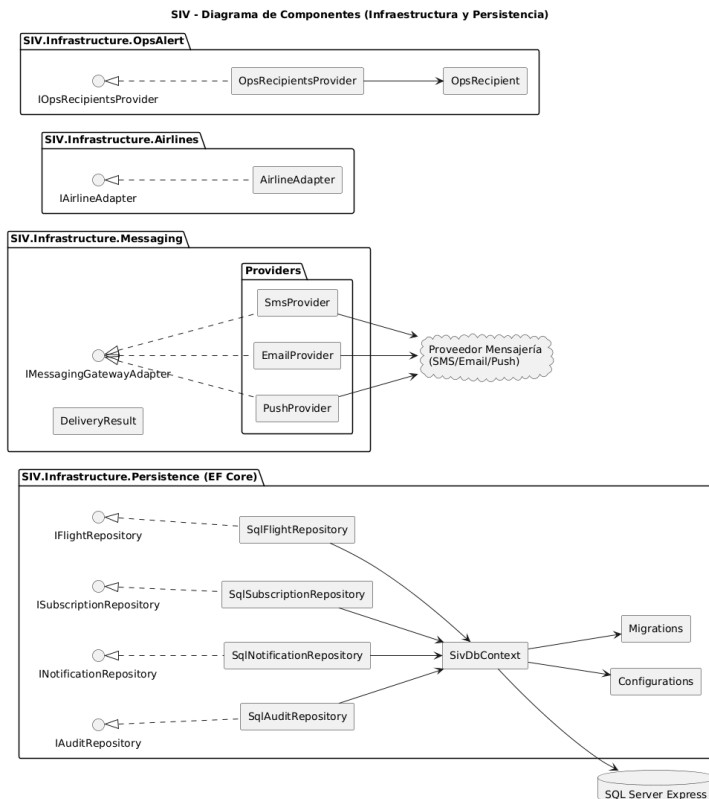


Application layer

SIV - Diagrama de Componentes (Capa Aplicación) con Domain Events (MVP)



Infrastructure layer:



En el MVP, las interfaces se mantienen representadas en Infraestructura como simplificación documental/organizativa, mientras los adaptadores concretos (p.ej. `SqlSubscriptionRepository`, `SqlFlightRepository`, `EmailProvider`, `SmsProvider`) se implementan en la misma capa.

Evolución (Escalamiento / Integración futura):

Para soportar reutilización del núcleo y minimizar acoplamiento, en una fase posterior los puertos (interfaces) se **moverán** a la capa de **Aplicación** (o `Application.Abstractions/Ports`), dejando Infraestructura exclusivamente para implementaciones/adaptadores.

Motivo: esto permite que el núcleo (dominio + casos de uso) pueda ser consumido por otros sistemas o despliegues sin arrastrar dependencias de infraestructura (EF Core, SQL Server, proveedores de mensajería), manteniendo Infra como “plug-in”.

3.2 Vista de procesos

Esta vista detalla los múltiples procesos que ocurren dentro del sistema elementos que influyen los componentes de este permitiendo a los servicios de comunicación (APIS) no solo conectar sino permitir la actualización de elementos y realización previamente colgadas en hilos, efectuando la realización de “x” hilo operativo, finalización y serie de este previniendo de esta manera que permanezcan hilos colgados que ya no serán efectuados o que en su defecto cumplieron su ciclo operativo.

Flujos del sistema:

- **Recepción de lotes** - la aerolínea hace envío de los lotes el sistema mantiene vivo este hilo de manera asincrónica mediante la Api de comunicación, esperando de esta manera el envío de lotes de los vuelos que circularan en el aeropuerto provenientes de dicha aerolínea.
- **Gestión de cambios** - el sistema permanece a la espera de manera asincrónica para recibir los cambios de los estados de los diversos vuelos que vienen de x aerolínea, permitiendo así la actualización en “real-time” de los cambios de estados que ocurren en dichos vuelos.
- **Autenticación de usuarios internos (operaciones)** - el sistema mediante un login efectúa una validación de credenciales realizando así un proceso de autenticación y autorización de los elementos que puede realizar/ver dentro del sistema pertinente, gestionando así un token que mantiene vivía la sesión durante el flujo operativo de dicho usuario.
- **Suscripción de usuarios externos** – el sistema permite a los usuarios realizar un proceso de suscripción al vuelo de su interés realizando un proceso sincrónico y dando una respuesta a la acción realizada por dicho usuario.
- **Envío de notificaciones** - el sistema envía notificaciones a los usuarios interesados mediante la programación y espera asincrónica gestionando de esta manera respuesta de éxitos/colgadas /fallos en el envío de dichas notificaciones o alertas en caso de tratarse del equipo de operaciones.
- **Cancelación de suscripciones** – el sistema permite el flujo sincrónico del proceso de cancelación de un seguimiento/suscripción de un vuelo al que el usuario se encontrara previamente suscripto.

Concurrencia

El sistema permitirá el flujo de múltiples tareas en paralelos garantizando la realización de diversas tareas de manera eficaz sin que afecte a otros usuarios a la realización de otra tarea.

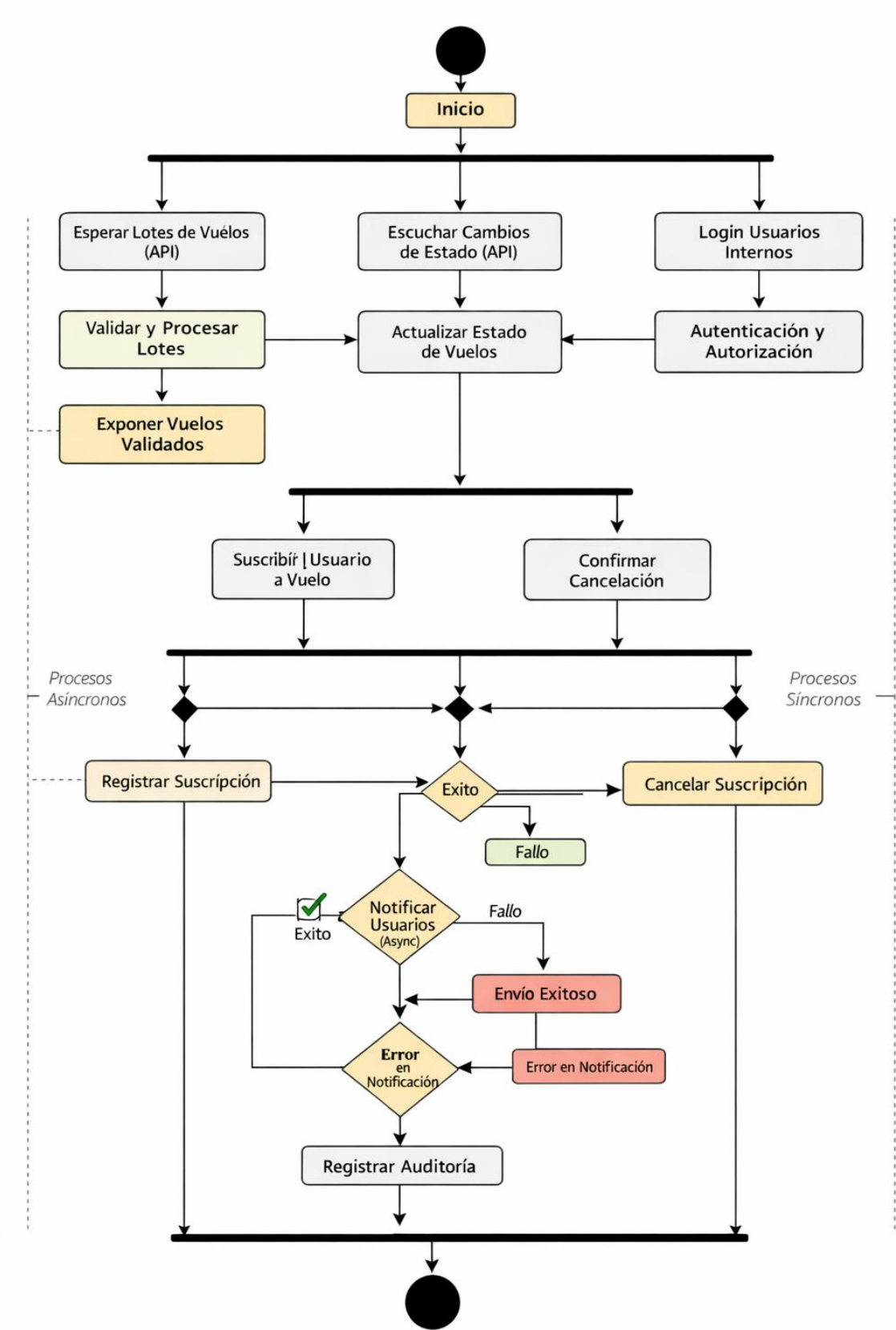
- **Recepción de lotes y actualización de estados** – se deberá permitir la recepción de lotes de vuelo por parte de las aerolíneas de manera sincrónica sin afectar dichas recepciones a vuelos ya establecidos por esas aerolíneas y que están siendo seguidos por x cantidad usuarios,
Permitiendo a su vez la actualización asincrónica de dichos estados de vuelo o resolución de incongruencias por parte de la aerolínea sin que afecte entonces a vuelos provenientes del mismo lote que se encuentren ya circulando de manera pública.
- **Seguimiento de vuelos** – deberá poder ocurrir de manera paralela a la actualización de vuelos permitiendo entonces a los usuarios realizar seguimientos con información exacta, aunque se haya producido un cambio de estado proveniente de la aerolínea al momento de este realizar una suscripción, a su vez permitirá entonces el flujo de múltiples usuarios que realicen la opción de seguimiento del mismo vuelo.
- **Cancelaciones de seguimiento** - este proceso debe poder ocurrir sin que afecte a otros procesos o que a su vez deba esperar una respuesta por uno de los procesos mencionados antes, realizando así su tarea de manera sincrónica.

Eventos

Diversos eventos dentro del sistema desencadenan una acción automática manejada sin intervención de un usuario.

- Validación de cumplimiento de parámetros mínimos para permitir el ciclo operativo de un vuelo.
- Exponer vuelos recibidos y validados para el seguimiento de usuarios externos (pasajeros/visitantes).
- Cambios de estados operativos por parte de la aerolínea.
- Suscripciones de usuarios – este evento provoca que el usuario se agregado a la lista de interesados del vuelo en cuestión.
- Cambios por parte del cuerpo operativo – producción procesos y registro de auditorías.

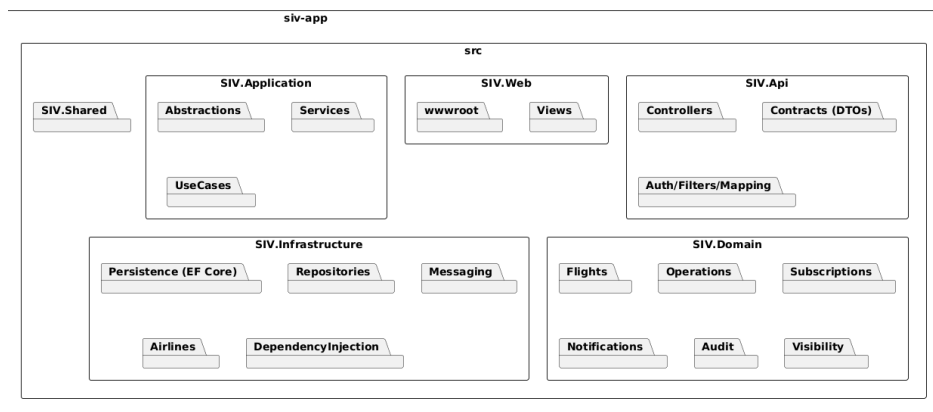
Diagrama de actividades:



3.3 Vista de desarrollo

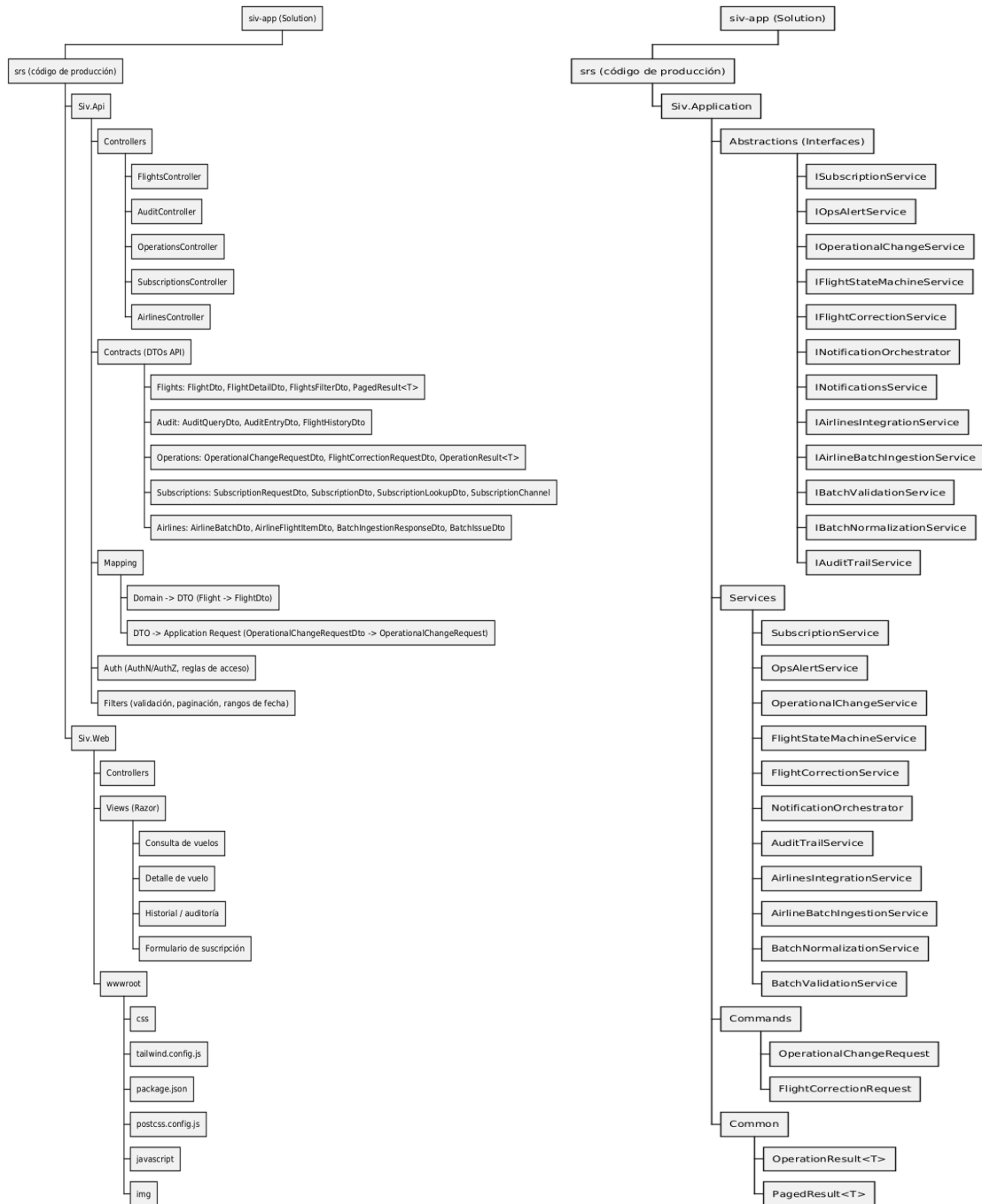
La organización modular del proyecto es la parte fundamental de esta vista y representativa del SIV, mostrando la disposición del código fuente, herramientas utilizadas en la configuración y desarrollo de los entornos de ejecución.

➤ Estructura del código:



➤ Carpetas:

Estructura de las carpetas principales del Sistema de información de vuelos

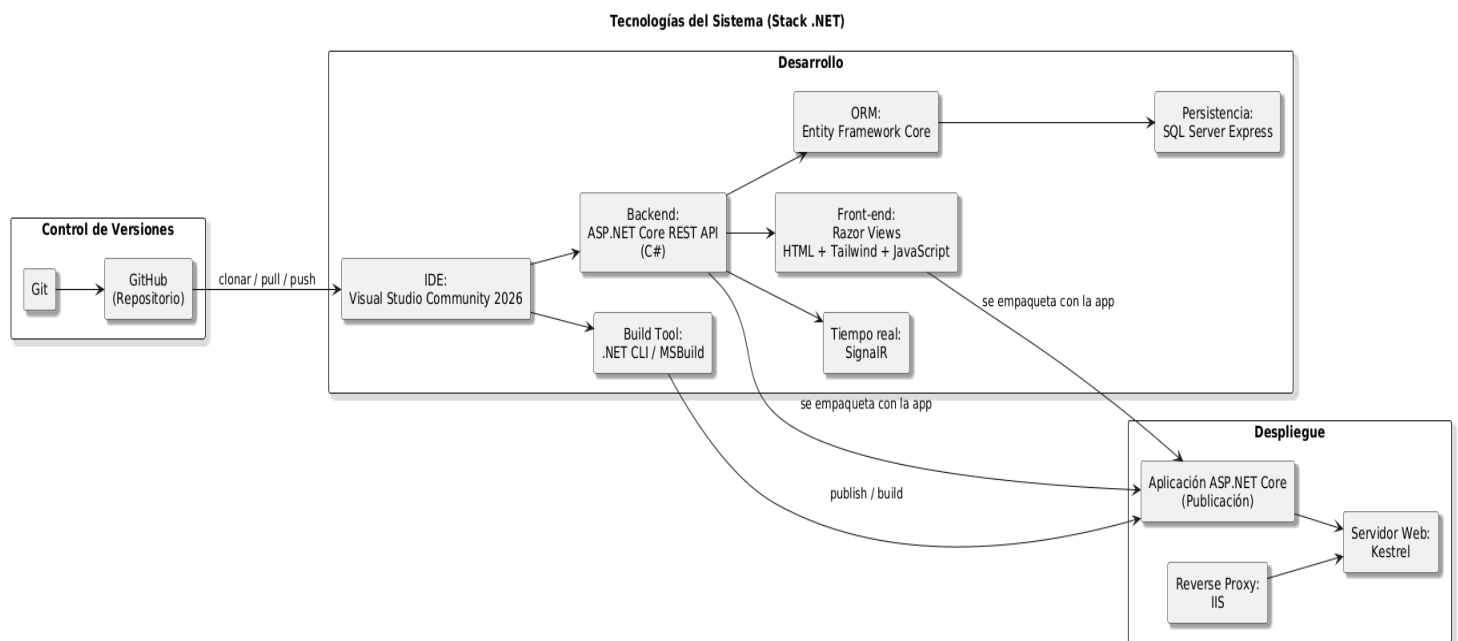


➤ Tecnologías por utilizar

Las tecnologías por utilizar en el desarrollo del MVP del Sistema de Información de Vuelos (SIV) son las siguientes:

- **Backend:** Ecosistema de .Net empleando C# específicamente Asp.net core Rest Api, SignalR
- **Front-end:** Razor Views combinando los elementos de: HTML, Tailwind y JavaScript.
- **Persistencia:** SQL Server Express, Entity Framework core como ORM (Object Relational Mapping).
- **Build tool:** .NET Cli/ MSBuild
- **Servidor:** Kestrel y IIS (Internet information services)
- **IDE:** Visual Studio Community 2026
- **Control de versiones:** Github/Git

Diagrama representativo



3.4 Vista de despliegue

En la vista presente se explica la infraestructura que implementan el sistema de información de vuelos (SIV) explicando así mis en que nodos corre el sistema, como se conectaran dichos nodos, que se instala en cada uno de estos nodos, etc. Así mismo, esta vista busca proporcionar una consulta/muestra al servicio de servidores que está previsto ejecutarse dentro del sistema, así como los elementos de nube y contenedores.

Los siguientes elementos detallan la infraestructura, servidores y elementos de la nube a utilizar para el desarrollo del sistema de información de vuelos.

Entorno de ejecución:

Entorno de desarrollo – este entorno será usado para el desarrollo y prueba del sistema logrando conseguir un MVP para el lanzamiento y prueba final del mismo, brindando acceso a la base de datos local y servicios simulados con fines de testeo del sistema.

Entorno de producción - este entorno representa el lanzamiento futuro de este sistema de información de vuelos llevando a unos de operativo real con acceso al público, integraciones reales con aerolíneas reales e instalaciones en aeropuertos reales, además de una auditoria activa.

Nodos de despliegue:

Nodo 1 – Cliente Web (usuarios) este nodo representa el navegador de cada usuario siendo dichos usuarios el personal de operaciones o su defecto usuarios externos que accedan a la plataforma del aeropuerto para verificar estados operarios de vuelos. Este brinda un acceso vía “HTTPS”.

Nodo 2 – Servidor web este nodo representa el servidor de aplicaciones en este primer escenario siendo un servidor local para pruebas unitarias de MVP siendo dicho servidor “IIS + Kestrel” con un run time en .Net (Asp.net core) aquí se despliegan los elementos de la aplicación web (razor Views), la API REST y el Signal Hub para la actualización en tiempo real.

Nodo 3 – Base de datos este se encuentra en SQL Server Express de acceso interno por el equipo de operaciones y empleando un ORM (Entity Framework) para el mapeo de las tablas de la base de datos.

Nodo 4 – Servicios externos de mensajería este nodo de proveedor es el encargado de la comunicación asincrónica y envío de notificaciones siendo el proveedor: SMS, Email o One Signal.

<i>Origen</i>	<i>Destino</i>	<i>Protocolo</i>	<i>Propósito</i>
<i>Navegador</i>	Servidor SIV	HTTPS	Consultas y operaciones
<i>Navegador</i>	SignalR Hub	WebSocket	Actualización en tiempo real
<i>Servidor SIV</i>	SQL Server	T-SQL	Persistencia
<i>Servidor SIV</i>	OneSignal/SMS/Email	HTTPS	Envío de notificaciones

Disponibilidad y escalabilidad:

El sistema de información de vuelos gracias a las tecnologías y métricas con las cuales será realizado tiene previsto una serie de elementos pertinente a la escalabilidad y disponibilidad de este:

- El servidor del SIV podrá escalar verticalmente
- El acceso concurrente es soportado por ASP.NET Core
- SignalR permite sincronización sin polling
- El sistema tolera múltiples usuarios simultáneos
- Fallos en mensajería no afectan la operación central

La organización de esta arquitectura y métricas hace que este sistema sea compatible al momento de ser desplegados en la nube con varios entornos entre los cuales están:

- AWS
- Azure
- Google Cloud Platform

Pudiendo integrarse entonces con el que cumpla el costo beneficio de terminando en el momento del requerimiento de instalación y despliegue masivo de dicho sistema, permitiendo así que los componentes del sistema se puedan mapear de la siguiente manera:

- Instancias de cómputo para el servidor de aplicaciones
- Base de datos gestionada (SQL Server administrado)
- Servicios de mensajería gestionados
- Balanceadores de carga para alta disponibilidad

Esta Compatibilidad permite:

- Escalabilidad horizontal del servidor de aplicaciones
- Alta disponibilidad
- Mantenimiento simplificado
- Separación clara entre entornos (dev / test / prod)

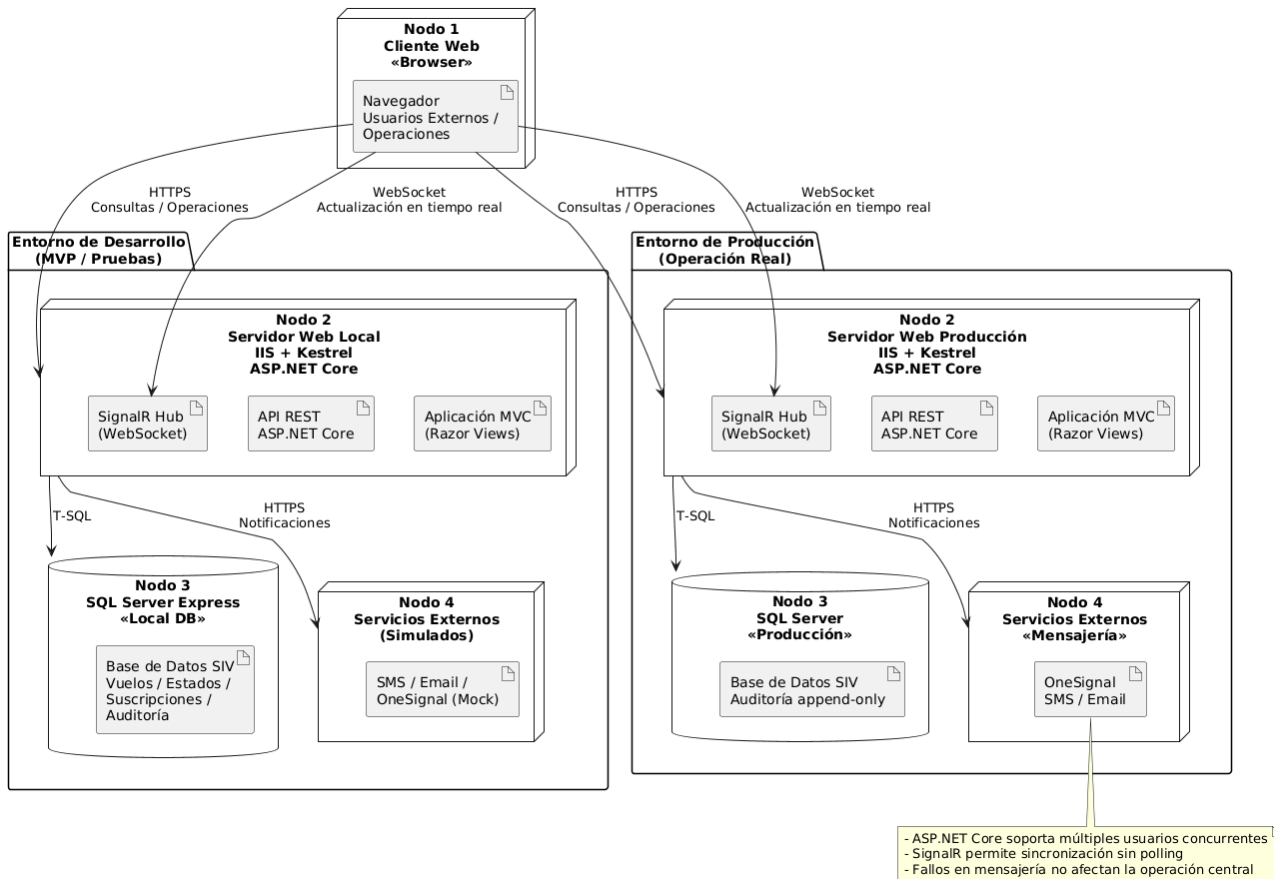
Contenedores:

Los contenedores de este sistema están basados en dos grandes bloques que permiten la integración futura de estos para la producción final del sistema.

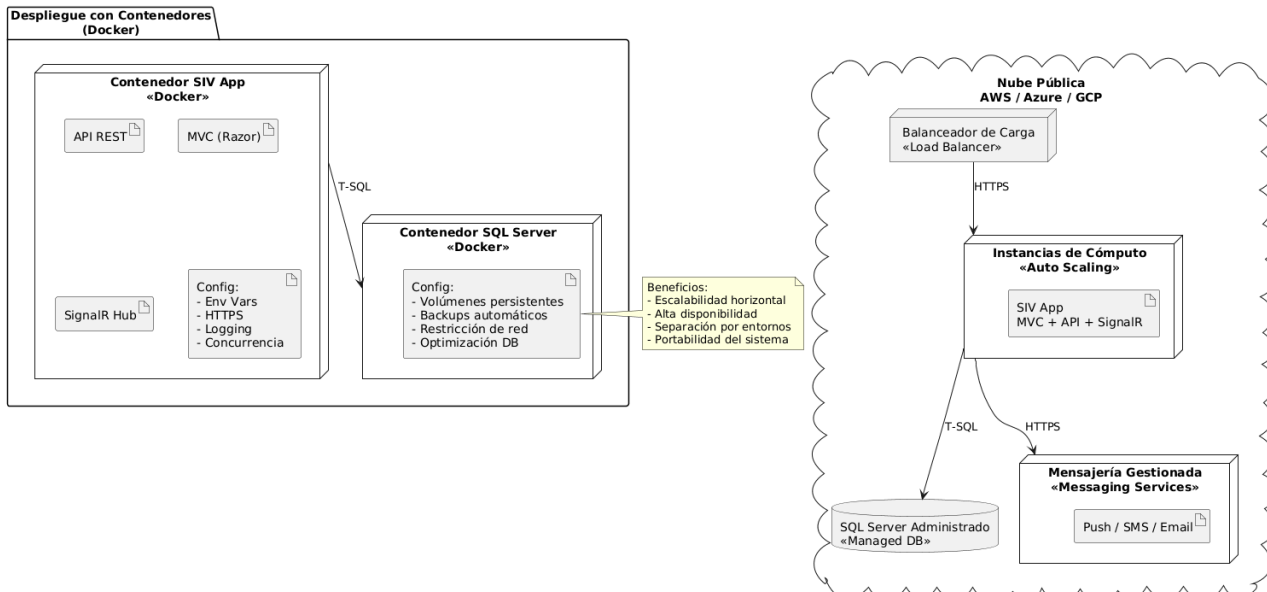
- Contenedor del servidor de aplicaciones:
 - Contiene:
 - API REST
 - Aplicación MVC
 - SignalR
 - Configuración:
 - Variables de entorno para conexiones
 - HTTPS habilitado
 - Logging centralizado
 - Manejo eficiente de memoria y concurrencia
- Contenedor de Base de datos:
 - Configuración:
 - Persistencia de datos mediante volúmenes

- Backups automatizados
- Restricción de acceso por red
- Optimización de índices y consultas

SIV (AeroVeloz) - Diagrama de Despliegue MVP



SIV (AeroVeloz) - Despliegue Futuro (Nube y Contenedores)

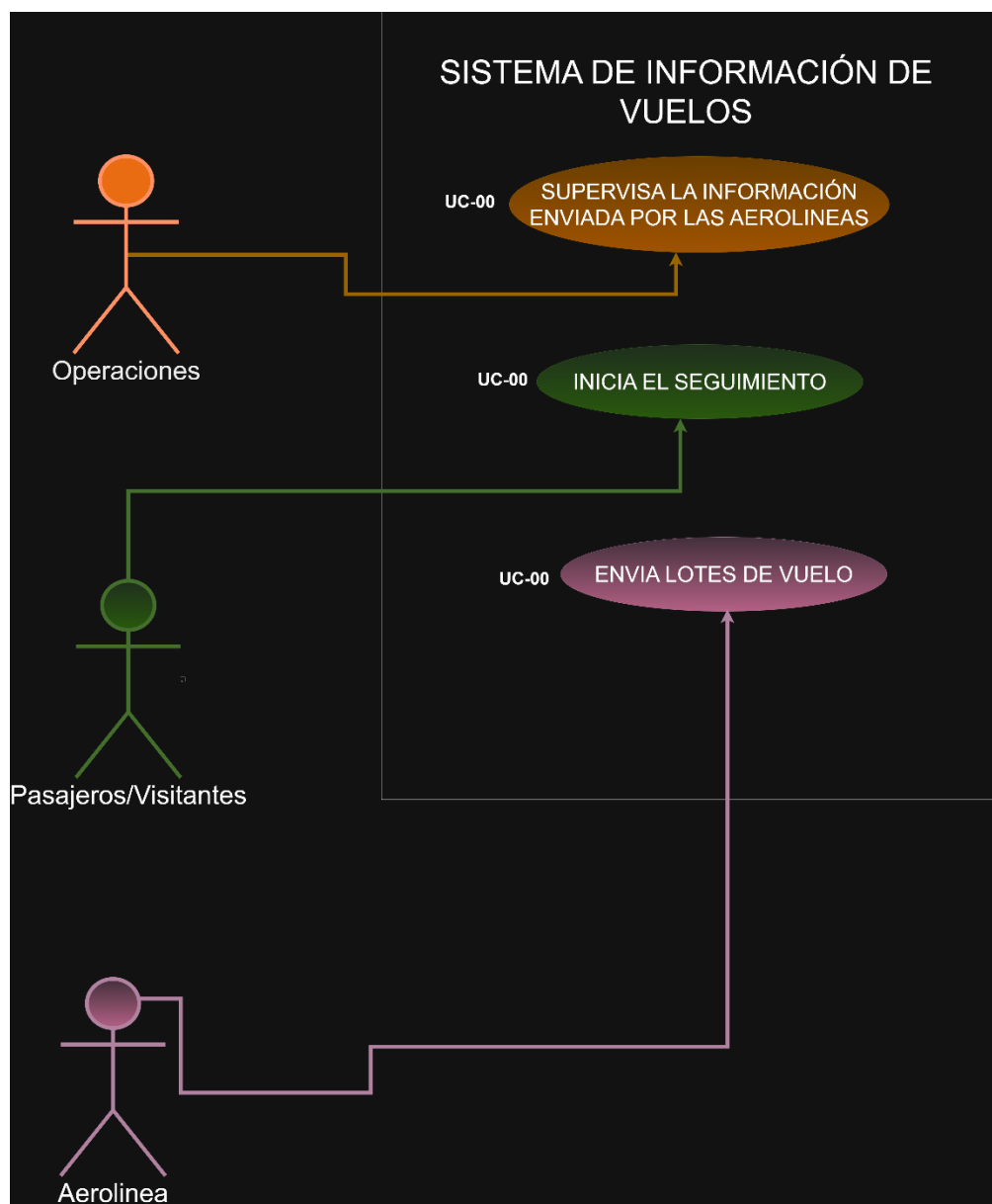


4. Casos de uso

En este apartado se muestran los diversos actores mostrando como interactúan con el Sistema de Información de Vuelos mostrando las acciones que estos realizan dentro del sistema y los escenarios pertinentes de cada uno.

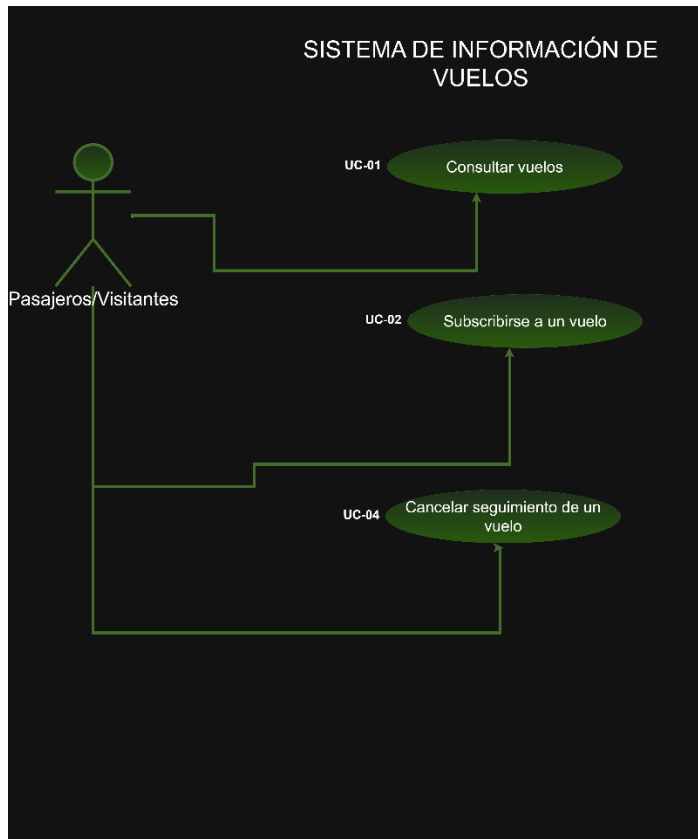
Diagrama de casos de uso

Caso de uso 1.

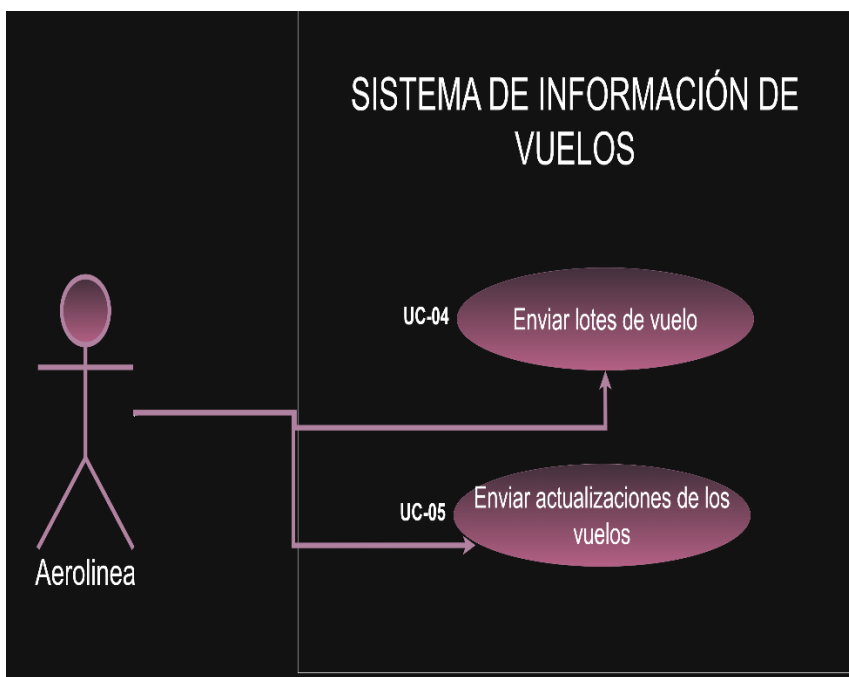


4.1 Caso de uso dependientes

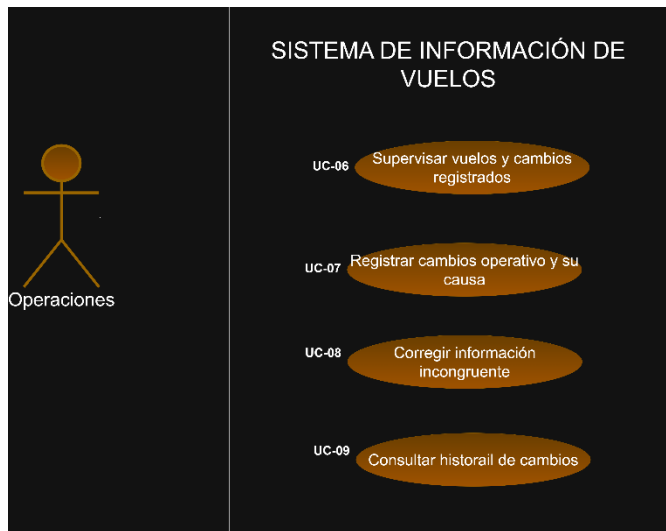
Actor externo: Pasajero/visitante



Actor externo: Aerolinea



Actor interno: Operaciones



4.2 Escenarios principales:

1.Consultar vuelos:

El usuario externo (pasajero/visitante) ingresa mediante su aplicativo correspondiente buscando su vuelo mediante criterios públicos (número de vuelo, aerolínea, fecha, etc), para ver el estado operativo vigente del vuelo, el sistema a su vez presenta dicha información de manera consistente. En caso de no existir el vuelo se informa al usuario externo.

2. Suscribirse/seguir vuelo:

El usuario externo mediante su aplicativo selecciona el vuelo de interés solicitando a su vez que desea “seguir/suscribirse” al vuelo, proporciona solo datos mínimos necesarios (sin requerir creación de cuenta) quedando así registrada la suscripción y el vuelo pasa a estar marcado como “seguido”.

3. Cancelar seguimiento:

El usuario externo solicita cancelar el seguimiento del vuelo y el sistema registra dicha cancelación sacando entonces a este usuario de la lista de interesados del vuelo en cuestión.

4. Enviar lote de vuelos/actualizaciones:

La aerolínea envía un lote de vuelos programados y/o cambios, el sistema recibe el lote si el lote no cumple con la información mínima se avisa al equipo de operaciones.

5. Supervisar vuelos y cambios:

El equipo de operaciones consulta listado/estado operativo de vuelos que fueron enviados por las diversas aerolíneas, revisando así cambios registrados y trazabilidad, si encuentra incongruencias debido aviso por parte del sistema o consulta manual corrige dicha incongruencia de vuelo y el sistema guarda los elementos de cambios y originales para auditoria.

5. Componentes clave

Esta sección detalla los componentes lógicos fundamentales del SIV, agrupando las clases y servicios definidos en la vista de desarrollo en bloques funcionales que garantizan la operatividad del sistema.

5.1. Módulos del Sistema

Basado en la arquitectura de capas y los servicios principales identificados, el sistema se compone de cuatro motores lógicos:

- **Motor de Gestión del Ciclo de Vida del Vuelo (Core Flight Engine):**
 - Componentes: Agrupa FlightQueryService y FlightStateMachineService.
 - Responsabilidad: Centraliza la lógica de negocio para las transiciones de estado (A tiempo, Retrasado, Cancelado). Garantiza que solo exista un estado vigente por vuelo y aplica las reglas de negocio de las 48 horas para las consultas públicas.
 - Interacción: Es consumido directamente por el FlightsController.
- **Módulo de Ingesta y Validación (Ingestion Pipeline):**
 - Componentes: Integra AirlineBatchIngestionService y BatchValidationService.
 - Responsabilidad: Procesa de manera asíncrona los lotes de datos enviados por las aerolíneas. Normaliza formatos, valida la integridad de la información y detecta incongruencias que requieren intervención del equipo de Operaciones.

- Sistema de Notificaciones y Suscripciones (Notification Dispatcher):
 - Componentes: SubscriptionsService, NotificationsService y la integración con OneSignal.
 - Responsabilidad: Gestiona la lista de interesados (suscripciones) por vuelo. Ante un evento de cambio de estado significativo, publica un evento de dominio y ejecuta handlers internos para, auditoria o notificaciones.
- Componente de Auditoría Inmutable (Audit Logger):
 - Componentes: AuditService y AuditController.
 - Responsabilidad: Intercepta todas las operaciones de escritura (cambios de estado, correcciones operativas) y las persiste en un formato *append-only* en SQL Server, asegurando la trazabilidad histórica requerida.

5.2. Interfaces y Contratos

Para mantener el desacoplamiento entre las capas de Aplicación e Infraestructura, el sistema expone las siguientes interfaces clave definidas en el diseño de clases:

Interfaz	Implementación / Responsabilidad
ISubscriptionRepository	Abstrae el acceso a datos de suscripciones, permitiendo operaciones como GetActiveByFlight sin depender directamente de SQL Server.
IFlightRepository	Contrato para la persistencia de vuelos y actualización de estados operativos.
IAuditRepository	Garantiza que los logs de auditoría sean escritos, pero nunca modificados o eliminados.
INotificationAdapter	Interfaz genérica que permite al sistema enviar mensajes sin conocer los detalles técnicos de OneSignal o proveedores de SMS.

5.3. Protocolos de Comunicación

El intercambio de información entre los componentes y actores externos se rige por los siguientes protocolos estándar:

- HTTP/HTTPS (REST API): Utilizado para todas las interacciones síncronas entre los clientes (Navegadores/Postman) y el backend (FlightsController, OperationsController). Se emplean verbos estándar (GET, POST, PUT) y formato JSON.
- SignalR (WebSockets): Implementado para la "actualización y sincronización en tiempo real" de las pantallas operativas, permitiendo que el personal de operaciones vea los cambios de estado sin necesidad de recargar la página.
- T-SQL: Protocolo de comunicación interna entre la capa de Infraestructura (Entity Framework Core) y el motor de base de datos SQL Server Express.
- Procesamiento Asíncrono (Internal Threads): Utilizado en la recepción de lotes de aerolíneas para no bloquear el hilo principal de la aplicación durante cargas masivas de datos.

6. Decisiones arquitectónicas

Durante el diseño del sistema se tomaron decisiones fundamentales para asegurar que el SIV represente un sistema que cumpla no solamente con requerimientos funcionales sino también con elementos no funcionales asegurando la mantenibilidad, seguridad, escalabilidad y la eficiencia operativa del SIV, lo que llevo a la selección de una arquitectura en capas (Layered Architecture), el uso del patrón de diseño MVC – Model-View-Control y la utilización de las tecnologías de .Net que permita el cumplimiento de lo antes mencionado. Así mismo, se definido el uso de los Domain – Event (in-process) para el manejo de los eventos y la implementación futura de event-driven para la alta de manda y despliegue masivo del SIV quedando claro el uso del Domain-event para el MVP del proyecto.

- Backend en .NET (Asp.net core Rest) + Front-end Razor Views: Se define un backend REST con ASP.NET Core y una capa UI con Razor Views, permitiendo una solución web coherente (MVP) y extendible.

- Persistencia SQL Server + Entity Framework Core (ORM):
Se seleccionó SQL Server (Express para MVP) con EF Core para estandarizar el acceso a datos y permitir evolución hacia un SQL Server administrado en producción.
- Auditoría inmutable (Audit Trail append-only) como requisito crítico:
Toda operación relevante produce un registro obligatorio y los logs de auditoría se almacenan en un esquema inmutable (append-only) para reconstrucción histórica y control.
- Notificaciones proactivas con proveedor externo (OneSignal / SMS / Email):
Se externaliza el envío de notificaciones usando proveedores, con adaptadores para evitar dependencia directa del proveedor.
- Actualización en tiempo real con SignalR:
Se integra SignalR para sincronización real-time de pantallas operativas sin recargar la página y evitando polling.
- DTOs para intercambio entre capas y reducción de exposición del dominio:
Se usan Data Transfer Objects para transferir datos sin exponer entidades de dominio directamente, aumentando claridad y seguridad del modelo.
- Despliegue en IIS + Kestrel y compatibilidad con nube y contenedores:
Se definió un despliegue inicial en IIS + Kestrel para MVP y compatibilidad futura con AWS/Azure/GCP y contenedores (Docker) para escalabilidad/portabilidad.

Justificación

- **Mantenibilidad y modularidad:** Las capas + MVC + DTO + repositorios permiten modificar UI, reglas de negocio o persistencia con impacto controlado, reduciendo acoplamiento.
- **Fiabilidad y trazabilidad operativa:** La auditoría append-only asegura evidencia histórica y reconstrucción de cambios, requisito clave en operación aeroportuaria.
- **Escalabilidad y evolución futura:** La compatibilidad con nube (compute, DB administrada, mensajería gestionada, balanceador) permite escalar horizontalmente cuando el despliegue sea masivo.
- **Rendimiento:** La regla de 48 horas limita volumen consultado y reduce presión sobre la base de datos; SignalR evita polling y reduce tráfico repetitivo.
- **Uso de Eventos de Dominio (In-Process):** Se implementan eventos en memoria para desacoplar las notificaciones y la auditoría sin añadir infraestructura compleja, cumpliendo con el MVP y facilitando una futura migración a microservicios.
- **Operación segura:** El control por roles y la separación de responsabilidades reducen superficie de error y evitan que usuarios externos ejecuten acciones operativas

7. Requisitos no funcionales

Seguridad:

- **Comunicación segura** - el sistema debe operar bajo HTTP/HTTPS para las interacciones cliente-servidor y mantener comunicaciones seguras en producción.
- **Control de acceso por roles (AuthN/AuthZ)** - los perfiles limitan funcionalidades; usuarios externos solo consultan información y operaciones internas requieren permisos.

- Trazabilidad obligatoria - toda modificación crítica debe generar un registro con actor, fecha y acción, garantizando supervisión y control.

Trazabilidad y auditabilidad:

- Auditoría inmutable (append-only) - los logs de auditoría deben ser inalterables, permitiendo reconstruir el historial completo de cambios operativos.
- Auditoría como componente transversal - el sistema registra eventos relevantes (cambios de estado, correcciones, notificaciones) para inspección posterior. Para el MVP los eventos relevantes se implementan como Eventos de Dominio internos (in – process) mediante delegados/handlers en la capa de Aplicación. En fase de alta demanda, estos podrán evolucionar a un esquema event-driven desacoplado (pub/sub/cola) sin cambiar el núcleo de dominio.”

Rendimiento:

- Optimización de consultas públicas - para proteger el desempeño, las consultas visibles a usuarios externos se restringen a una ventana de 48 horas (pasadas y futuras) respecto al servidor, reduciendo carga en BD y mejorando respuesta.
- Comunicación real-time sin polling - la sincronización en tiempo real se realiza con SignalR/WebSockets, evitando recargas/polling constante y mejorando eficiencia.

Escalabilidad:

- Evolución gradual del sistema - la arquitectura modular permite extender el sistema con nuevas integraciones sin comprometer el núcleo.
- Compatibilidad con nube - el despliegue puede mapearse a nube pública (AWS/Azure/GCP) con cómputo, base de datos administrada, mensajería gestionada y balanceadores para alta disponibilidad.

- Canales externos desacoplados: el sistema integra mensajería externa (SMS/Email/Push) como canal de salida, evitando que el core dependa de la implementación del proveedor.

Usabilidad:

- Acceso web desde navegadores modernos - el sistema se consume mediante una aplicación web (MVC) accesible desde navegadores modernos para consultas y operación.
- Consistencia informativa multicanal - el sistema funciona como fuente centralizada y consistente, reflejando cambios de estado de forma inmediata y proactiva.

Mantenibilidad:

- Separación estricta de responsabilidades presentación, aplicación, dominio y persistencia tienen roles definidos, reduciendo acoplamiento y facilitando mantenimiento.
- Patrones establecidos uso de Service Layer, Repository Pattern, DTO y Audit Trail para hacer el sistema extensible y claro.

8. Riesgos técnicos y mitigaciones

Riesgo Técnico	Impacto Potencial	Estrategia de Mitigación
Latencia en la Ingesta de Datos	Si el procesamiento de lotes grandes de las aerolíneas bloquea el hilo principal, la interfaz web podría congelarse para los usuarios operativos.	Mitigación: Implementación de procesamiento asíncrono y colas en memoria para la ingesta de lotes, asegurando que la recepción de datos no bloquee la respuesta de la API.
Fallo en el Proveedor de Mensajería	Si OneSignal o el servicio de correo cae, las notificaciones críticas de cambios de estado podrían perderse.	Mitigación: Diseño desacoplado mediante interfaces (INotificationAdapter). El sistema registrará los intentos fallidos en el log de auditoría para permitir un reintento manual o automático posterior.
Desincronización de SignalR	En caso de micro-cortes de red, el cliente WebSocket podría desconectarse y dejar de recibir actualizaciones en tiempo real.	Mitigación: Configuración de reconexión automática en el cliente de SignalR y un botón de "Actualización Manual" en la interfaz operativa como respaldo
Crecimiento Exponencial de Logs	El esquema de auditoría inmutable (<i>append-only</i>) puede generar un volumen masivo de datos que degrade el rendimiento de la base de datos.	Mitigación: Indexación optimizada en la tabla de auditoría por FlightId y Date, y la posibilidad futura de archivar logs antiguos (> 1 año) en un almacenamiento en frío (<i>Cold Storage</i>).
Inconsistencia de Datos Externos	Las aerolíneas podrían enviar datos con formatos erróneos o códigos de vuelo duplicados.	Mitigación: Módulo de BatchValidationService estricto que actúa como cortafuegos (<i>Firewall</i> de datos), rechazando registros inválidos antes de que toquen la capa de dominio.

9. Conclusiones

El diseño arquitectónico del Sistema de Información de Vuelos (SIV) que se muestra en este documento proporciona una base firme, actual y conforme a las normas del sector para solucionar la cuestión de la desincronización informativa en los contextos aeroportuarios.

Hemos conseguido lo siguiente gracias a la adopción del modelo MVC en ASP.NET Core y a una rigurosa Arquitectura en Capas:

Asegurar la integridad: El SIV es una "Fuente Única de Verdad" confiable y auditable debido a la centralización de la lógica de estados y al empleo de auditoría inmutable.

Maximizar la experiencia: La incorporación de SignalR y las notificaciones proactivas cambian la experiencia del viajero, al pasar de un sistema de "búsqueda continua" a uno basado en la "recepción de información".

Garantizar el progreso: El desacoplamiento a través de interfaces y DTOs posibilita la expansión del sistema, simplificando la integración futura con aplicaciones nativas móviles o nuevos proveedores de datos sin necesidad de reescribir el núcleo de la empresa. En resumen, esta arquitectura no solo satisface las necesidades funcionales del MVP, sino que además muestra un sistema escalable, confiable y fácil de mantener, capaz de sostener la operación esencial de un aeropuerto contemporáneo.